



Figure 1: Grails folder

new terminal and run a command in it, which will be similar to the one that follows:

```
source "/home/username/.gvm/bin/gvm-init.sh"
```

After you run the above script, you'll be able to use the GVM to install Grails.

3. *Installing Grails:* Run the following command to install

```
gvm install grails
```

To install a specific version of Grails (say, version 2.3.2), you can run the following command.

```
gvm install grails-2.3.2
```

4. *Testing your installation:* The simplest way to test your installation of Grails is to use it to create a project. Use the following command to create a new Grails project.

```
mkdir test
cd test
grails create-app test
A sample project
```

The above command creates an application for you. In our above example, we used the name 'test' for our project. The really nifty thing about Grails (or any other Web framework) is that it provides a boilerplate application, which is ready



Figure 2: Grails screenshot

to be used with minimal changes. Since Grails follows a 'code by convention' paradigm, you can build an application very quickly if you get familiar with the Grails standard conventions. The best way to figure out some of the more obvious conventions used in Grails is to look at the boilerplate application created for you. Figure 1 gives a screenshot of the files and folders Grails generates for you.

For your reference, here is a breakdown of the folder structure of Grails.

- grails-app: Top level directory for Groovy sources
- conf: Configuration sources
- controllers: Web controllers - the 'C' in MVC
- domain: The application domain
- i18n: Support for internationalisation (i18n)
- services: The service layer
- taglib: Tag libraries
- utils: Grails specific utilities
- views: Groovy server pages - the 'V' in MVC
- scripts: Gant scripts
- src: Supporting sources
- groovy: Other Groovy sources
- java: Other Java sources
- test: Unit and integration tests

Now let's see Grails in action. To run the pre-built application, use the following commands.

```
cd test
grails run-app
```

At this point, Grails will install all the dependencies and do a couple of complex optimisations, which it displays on your screen. When it is ready, it will print the following message.

```
Server running. Browse to http://localhost:8080/test
```

At this point, you can fire up your browser and type that URL into the URL bar (*Pro tip:* if you're feeling a bit lazy, clicking *Control* on the link in the terminal will open the link in your default browser).

When you go to the URL, you'll be able to see a page like the screenshot given in Figure 2, which lists specifics about various plugins and your application's configuration details. If you do see this screen, you've successfully installed Grails for your development environment.

Warning: The *run-app* command is intended for *development* purposes only. It isn't meant to be used for production environments. In production, you could use Grails to build WAR (Web Application Archive) files and then deploy it on an Apache Tomcat server, for instance, because a lot of possible optimisations aren't supported by the Grails' run-app server.

GORM

GORM is Grails's object relational mapping (ORM) implementation. Under the hood, it uses Hibernate 3 (a